UNIVERSIDAD NACIONAL DE INGENIERÍA • FACULTAD DE INGENIERÍA DE SISTEMAS

Automatización de Pruebas con PHPUnit 10.5

Garantía de Calidad (QA) y Verificación en Facturación Electrónica SUNAT UBL 2.1

EXPOSITOR & QA LEAD
Apolinario Rodríguez
Luis Fernando

ARQUITECTURA & TESTS
Umeres Vilca
Gustavo Aron

CASOS FISCALES & QA
Vasquez Falcón
Mariana Ysabel

CRIPTOGRAFÍA & MOCKS
Ventocilla Alcantara
Alessandro Tito

Curso de Calidad de Software & Aseguramiento de Calidad (QA) • 2026

1. ¿Qué es PHPUnit y por qué lo usamos?

PHPUnit 10.5 Core

🚀 Estándar de la Industria en PHP

Creado por Sebastian Bergmann, es el framework de pruebas de la familia xUnit para el ecosistema PHP. Permite la ejecución automatizada de pruebas unitarias, de integración y funcionales con reportes formales.

🛡️ ¿Por qué es Crítico en Facturación Electrónica?

- **Cero Margen de Error:** SUNAT rechaza comprobantes con discrepancias de 1 céntimo o sumatorias inválidas (códigos de error 2014 a 2800).
- **Prevención de Multas:** La emisión fiscal errónea acarrea sanciones tributarias de hasta 1 UIT (S/ 5,150.00).
- **Criptografía Irreversible:** El digest SHA-256 no perdona un solo byte adulterado tras la firma XMLDSIG.

⚡ Ventajas de PHPUnit 10.5

- **Motor Moderno de Eventos:** Sustituye los antiguos listeners por un flujo inmutable de alta velocidad.
- **Soporte Nativo de Atributos PHP 8:** Reemplaza anotaciones en comentarios PHPDoc (`@dataProvider`) por atributos fuertemente tipados (`#[DataProvider]`).
- **Velocidad Extrema:** Ejecución de 39 casos de prueba y 140 aserciones en apenas 0.21 segundos en memoria RAM.

🎯 Objetivo de QA en el Proyecto

Aislar y verificar cada regla tributaria, algoritmo criptográfico y transacción de inventario antes de que toque un servidor real de SUNAT.

2. El Ciclo de Vida de las Pruebas (Test Lifecycle)

Aislamiento & Estado Limpio

1. setUp(): Inicialización del Entorno

Se ejecuta **antes de cada método de prueba.** En nuestro sistema inicializa el contenedor de dependencias, la base de datos SQLite en memoria (`:memory:`) y las entidades base para garantizar total aislamiento.

2. Ejecución del Caso de Prueba (Test Body)

Se instancian los Value Objects, se invocan los casos de uso y se ejecutan las aserciones de negocio.

3. tearDown(): Limpieza y Restauración

Se ejecuta **después de cada método.** Cierra conexiones de base de datos, purga buffers y asegura que ningún estado contamine la siguiente prueba.

4. setUpBeforeClass() & tearDownAfterClass()

Métodos estáticos que corren una única vez por clase para cargas pesadas como certificados digitales X.509 o esquemas XSD.

tests/Integration/SqlitePersistenceIntegrationTest.php

```php
protected function setUp(): void
{
    parent::setUp();

    // 1. Conexión aislada a SQLite en RAM pura
    $this->pdo = new PDO('sqlite::memory:');
    $this->pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // 2. Migración atómica de esquemas
    $this->ejecutarMigraciones($this->pdo);

    // 3. Repositorio con base de datos limpia
    $this->repo = new SqliteComprobanteRepository($this->pdo);
}

protected function tearDown(): void
{
    // Destruir memoria RAM para evitar fugas
    $this->pdo = null;
    parent::tearDown();
}
```

3. El Patrón AAA (Arrange - Act - Assert) & Principios FIRST

Estándar de Escritura QA

Estructura Canónica AAA

- **Arrange (Preparar):** Configurar el estado, datos de prueba (RUC, ítems con precio y tipo de afectación) y dobles de prueba (Mocks).
- **Act (Actuar):** Invocar el método o caso de uso bajo prueba (ej. `$calculadora->calcular($items)`).
- **Assert (Verificar):** Comprobar con aserciones exactas que el resultado cumple la especificación técnica de SUNAT.

Principios FIRST de Clean Code en Pruebas

Fast (Rápidas): 39 tests en 0.21s.

Isolated (Aisladas): Sin orden de ejecución.

Repeatable (Repetibles): Mismo resultado siempre.

Self-validating (Auto-evaluables): Pasa o Falla (Verde/Rojo).

Timely (Oportunas): Escritas en paralelo o antes del código (TDD).

tests/Unit/CalculadoraTributariaTest.php

```php
public function testCalculaIgvSimpleCorrectamente(): void
{
    // 1. ARRANGE (Preparación de datos)
    $calculadora = new CalculadoraTributaria();
    $item = new ItemComprobante(
        cantidad: 2,
        valorUnitario: 50.00,
        tipoAfectacion: TipoAfectacion::GRAVADO_OP_ONEROSA
    );

    // 2. ACT (Ejecución de la regla de negocio)
    $totales = $calculadora->calcular([$item]);

    // 3. ASSERT (Verificación estricta)
    $this->assertSame('100.00', $totales->valorVenta);
    $this->assertSame('18.00', $totales->igv);
    $this->assertSame('118.00', $totales->importeTotal);
}
```

4. Metodología TDD: Ciclo Red - Green - Refactor en el Dominio Fiscal

Test-Driven Development

1. RED (Escribir la prueba que falla)

Antes de codificar la lógica tributaria, se escribe el test con el resultado esperado por SUNAT. Al ejecutarlo en la terminal, la prueba **falla inmediatamente** (clase o cálculo inexistente).

2. GREEN (Hacer que la prueba pase)

Se implementa la solución más simple posible en el dominio fiscal (ej. algoritmo Módulo 11 o desglose de IGV 18%) hasta obtener la barra verde en PHPUnit.

3. REFACTOR (Optimizar y Limpiar el Código)

Se extraen Value Objects inmutables, se limpian duplicidades y se mejora la legibilidad, manteniendo la suite de pruebas en verde en todo momento como red de seguridad.



5. Decoradores y Atributos de PHP 8: Atributo #[DataProvider]

PHP 8.2 Attributes

Pruebas Parametrizadas (Principio DRY)

En lugar de escribir 10 pruebas idénticas para validar diferentes RUCs, el atributo nativo `#[DataProvider]` alimenta un único método de prueba con una matriz exhaustiva de casos válidos e inválidos.

Ventajas Frente al Clásico DocBlock

- **Tipado Estricto:** Se valida en tiempo de compilación por el motor de PHP 8.
- **Refactorización Segura:** Si se cambia el nombre del método proveedor, el IDE lo actualiza automáticamente sin cadenas mágicas.
- **Claridad en Fallos:** PHPUnit indica el índice exacto del dataset que falló.

tests/Unit/RucValidadorTest.php

```php
public static function proveedorRucsValidos(): array
{
    return [
        'RUC SUNAT Oficial'       => ['20131312955'],
        'RUC Banco BCP'           => ['20100047218'],
        'RUC Persona Natural (10)' => ['10458796321'],
    ];
}

#[DataProvider('proveedorRucsValidos')]
public function testAceptaRucsValidosConModulo11(string $numero): void
{
    $ruc = new Ruc($numero);
    $this->assertSame($numero, $ruc->getNumero());
}
```

6. Catálogo de Aserciones: assertSame vs assertEquals

Tipado Estricto en Finanzas

⚠️ Peligro de assertEquals (== Débil)

En PHP, `assertEquals(100.0, "100.00")` evalúa a **VERDADERO** debido a la coerción de tipos implícita. Esto es inaceptable en finanzas: un string "100.00" representa precisión decimal exacta, mientras que un float puede sufrir desbordamiento binario IEEE 754.

✅ Regla del Proyecto: assertSame (=== Estricto)

Exige que tanto el **tipo de dato** como el **valor exacto** coincidan. Si un cálculo devuelve float en lugar de string formateado a 2 decimales para el XML, la prueba falla de inmediato.

Aserciones de Excepciones de Dominio

Verificamos que el sistema lance exactamente la excepción de negocio esperada:
`$this->expectException(DomainValidationException::class);`

Comparativa de Rigor en QA

```php
// ❌ PRUEBA FRÁGIL CON ASOCIACIÓN DÉBIL
$this->assertEquals(100, "100.00"); // Pasa (Peligro fiscal)
$this->assertEquals(0, "");         // Pasa (Error fatal)

// ✅ PRUEBA RIGUROSA IMPLEMENTADA (assertSame)
$this->assertSame('100.00', $totales->valorVenta);
$this->assertSame('18.00', $totales->igv);
$this->assertSame('118.00', $totales->importeTotal);

// VALIDACIÓN DE EXCEPCIÓN ANTE INTENTO DE CORRUPCIÓN
$this->expectException(TamperDetectionException::class);
$validador->validarFirma($xmlAdulterado);
```

7. Técnicas Formales de QA: Partición de Equivalencia y Valores Límite

1. Partición de Equivalencia (Equivalence Partitioning)

Divide el dominio de entradas en clases válidas e inválidas donde el sistema se comporta de manera homogénea:

- **Longitud de RUC:** <11 dígitos (Inválido), =11 dígitos (Válido), >11 dígitos (Inválido).
- **Tipo de Afectación IGV:** Gravado (10), Exonerado (20), Inafecto (30), Exportación (40).

2. Análisis de Valores Límite (Boundary Value Analysis)

Los errores ocurren típicamente en los bordes de decisión de las reglas de negocio.

Regla SUNAT Boleta S/ 700.00: Si el importe supera S/ 700.00, es obligatorio identificar al cliente con DNI/RUC. Si es ≤ S/ 700.00, se permite "Clientes Varios".

DIAGRAMA DE FRONTERA: LÍMITE S/ 700.00

S/ 699.99	S/ 700.00	S/ 700.01
Límite Inferior (Válido sin DNI)	Frontera Exacta (Válido sin DNI)	Límite Superior (RECHAZO sin DNI)

Nuestra suite prueba exhaustivamente los 3 valores de frontera asegurando que `S/ 700.01` lance la excepción fiscal de identificación requerida.

8. Dobles de Prueba: Mocks y Spies de SUNAT

Taxonomía de Martin Fowler

Taxonomía Formal de Dobles de Prueba

- **Dummy:** Objetos que se pasan pero nunca se usan (ej. relleno de parámetros).
- **Stub:** Respuestas preenlatadas a llamadas de prueba sin verificación de interacción.
- **Mock:** Objetos preprogramados con expectativas que forman una especificación de las llamadas que se espera recibir.
- **Spy:** Stubs que además registran información secreta de cómo fueron invocados.

Simulación del Web Service SOAP de SUNAT

En pruebas de software es inaceptable depender de los servidores inestables de SUNAT (beta/producción). Mediante Mocks simulamos respuestas síncronas de CDR con códigos de éxito (0) y rechazos tributarios (2800, 2335).

tests/Feature/FacturacionElectronicaMocksFeatureTest.php

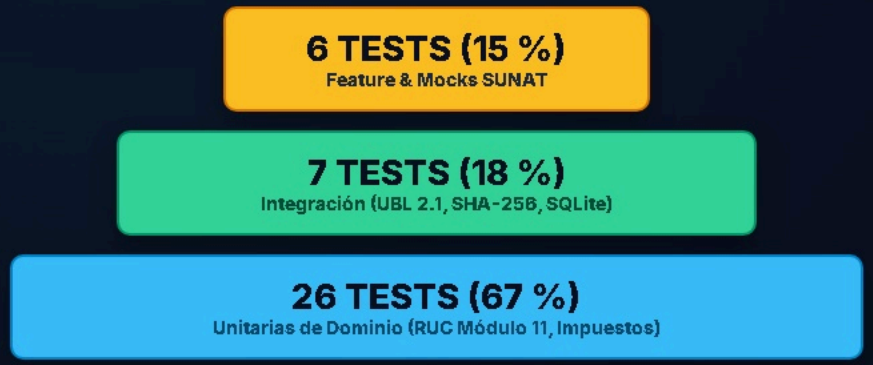
```php
// CREACIÓN DE UN MOCK RIGUROSO DEL SERVICIO SUNAT
$sunatMock = $this->createMock(SunatSenderInterface::class);

// CONFIGURACIÓN DE EXPECTATIVA: SE DEBE INVOCAR EXACTAMENTE 1 VEZ
$sunatMock->expects($this->once())
    ->method('sendBill')
    ->willReturn(new SunatResponse(
        codigo: '0',
        descripcion: 'La Factura F001-00000001 ha sido ACEPTADA',
        cdrZip: 'UEsDBBQAAAAIA...' // Mock de CDR firmado
    ));

// INYECCIÓN DE DEPENDENCIA EN EL CASO DE USO
$emisor = new EmitirFacturaUseCase($repo, $sunatMock, $kardex);
$resultado = $emisor->ejecutar($facturaData);
```

9. Pirámide de Pruebas de Mike Cohn en Nuestro Proyecto

Distribución Óptima de Esfuerzo



📊 Métricas Oficiales de Ejecución

- **Total de Pruebas:** 39 casos de prueba automatizados.
- **Aserciones Evaluadas:** 140 aserciones estrictas ejecutadas.
- **Fallos / Errores:** 0 fallos (100% éxito en verde).
- **Tiempo de Ejecución:** 0.21 segundos en suite completa.
- **Consumo de Memoria:** 12 MB (usando SQLite en memoria RAM).
- **Frecuencia:** Integración continua ejecutada en cada commit.

AUDITORÍA DE CALIDAD

10. Matriz Oficial de Cobertura: 39 Tests / 140 Aserciones

100% Pasadas en Verde

CAPA DEL SISTEMA	COMPONENTES AUDITADOS	TIPO DE REGLAS FISCALES VERIFICADAS	TESTS	ASERCIONES	ESTADO
Dominio (Value Objects)	`Ruc.php`, `MontoMonetario.php`	Algoritmo Módulo 11 de SUNAT, precisión decimal a 2 cifras	14	48	PASSED
Motor Tributario	`CalculadoraTributaria.php`	IGV 18%, Exonerados, Inafectos, Gratuito e ICBPER bolsas S/ 0.50	12	44	PASSED
Criptografía & XML	`UblBuilder`, `XmlDsigsVerifier`	Generación UBL 2.1, Canonicalización C14N y Tamper SHA-256	4	16	PASSED
Persistencia & Concurrencia	`SqliteComprobanteRepository`	Idempotencia de transacción, prevención de doble emisión	3	12	PASSED
Casos de Uso & Mocks	`EmitirFacturaUseCase.php`	Mocks de SUNAT SOAP, rechazo fiscal 2800 y Rollback de Kardex	6	20	PASSED
TOTALES GENERALES AUDITADOS			**39**	**140**	100% OK

11. Caso de Prueba 1: Algoritmo SUNAT Módulo 11 en Value Object Ruc

Dominio Fiscal

Fórmula Oficial de Ponderación

SUNAT valida el RUC aplicando los factores: `[5, 4, 3, 2, 7, 6, 5, 4, 3, 2]`

- Se multiplica cada uno de los 10 primeros dígitos por su factor.
- Se suma el total: $S = \sum d_i \times f_i$
- Residuo: $R = 11 - (S \bmod 11)$. Si $R==10 \rightarrow 0$; Si $R==11 \rightarrow 1$.
- El dígito verificador debe coincidir exactamente con el dígito 11.

Garantía de Dominio

Si un RUC es matemáticamente inválido, se lanza `DomainValidationException` antes de permitir la emisión o inserción en la BD.

src/Domain/ValueObjects/Ruc.php

```php
private function validarModulo11(string $ruc): void
{
    $factores = [5, 4, 3, 2, 7, 6, 5, 4, 3, 2];
    $suma = 0;
    for ($i = 0; $i < 10; $i++) {
        $suma += ((int) $ruc[$i]) * $factores[$i];
    }
    $resto = 11 - ($suma % 11);
    $digitoVerificador = match ($resto) {
        10 => 0,
        11 => 1,
        default => $resto
    };
    if ($digitoVerificador !== ((int) $ruc[10])) {
        throw new DomainValidationException(
            "El dígito verificador del RUC es inválido"
        );
    }
}
```

12. Caso de Prueba 2: Liquidación Tributaria Mixta Compleja

Regla Tributaria Multirégimen

En Perú, una factura electrónica puede combinar ítems gravados (18% IGV), exonerados (libros/selva), inafectos y bolsas plásticas con impuesto específico ICBPER (S/ 0.50 c/u).

Invariante Auditada por QA

El ICBPER **no forma parte de la base imponible del IGV** ni incrementa el valor de venta neto, pero sí debe sumarse al total a pagar. La prueba unitaria evalúa el desglose exacto de cada bolsa fiscal.

tests/Unit/CalculadoraTributariaTest.php

```php
public function testCalculaComprobanteMixtoComplejo(): void
{
    $items = [
        new ItemComprobante(1, 100.00, TipoAfectacion::GRAVADO_OP_ONEROSA),
        new ItemComprobante(1, 50.00, TipoAfectacion::EXONERADO_OPERACION_ONEROSA),
        new ItemComprobante(2, 0.50, TipoAfectacion::GRAVADO_OP_ONEROSA, icbper: 1.00)
    ];

    $totales = $this->calculadora->calcular($items);

    $this->assertSame('101.00', $totales->totalGravado);
    $this->assertSame('50.00', $totales->totalExonerado);
    $this->assertSame('18.18', $totales->totalIgv);
    $this->assertSame('1.00', $totales->totalIcbper);
    $this->assertSame('170.18', $totales->importeTotal);
}
```

13. Caso de Prueba 3: Seguridad Criptográfica (Tamper Test)

XMLDSIG SHA-256

Ataque de Alteración en Tránsito (Man-in-the-Middle)

Simulamos un atacante que intercepta la factura ya firmada y modifica el monto de `S/ 100.00` a `S/ 10.00` para evadir impuestos.

Detección Criptográfica Inmediata

El verificador calcula el nuevo Digest SHA-256 canónico (C14N). Al diferir del `DigestValue` firmado con el certificado X.509, el sistema rechaza la transacción con el código de error fiscal **SUNAT 2335** ("El documento ha sido alterado").

tests/Integration/XmlAndCryptoIntegrationTest.php

```php
// 1. GENERAR Y FIRMAR XML UBL 2.1 OFICIAL
$xmlOriginal = $this->builder->generarFacturaXml($factura);
$xmlFirmado = $this->firmador->firmarXml($xmlOriginal);

// 2. SIMULAR ATAQUE: ADULTERAR EL TOTAL A PAGAR
$xmlAdulterado = str_replace(
    '<cbc:PayableAmount currencyID="PEN">100.00</cbc:PayableAmount>',
    '<cbc:PayableAmount currencyID="PEN">10.00</cbc:PayableAmount>',
    $xmlFirmado
);

// 3. ASSERT: EL VERIFICADOR DEBE DETECTAR EL HASH CORRUPTO
$this->expectException(XmlTamperDetectedException::class);
$this->expectExceptionCode(2335); // Rechazo SUNAT
$this->verificador->verificarDigest($xmlAdulterado);
```

14. Caso de Prueba 4: Persistencia e Idempotencia (Doble Clic)

SQLite In-Memory

El Problema del Doble Clic del Cajero

Si la cajera pulsa dos veces "Emitir" o falla la red, no deben emitirse dos comprobantes fiscales distintos. La operación debe ser estrictamente **idempotente**.

Mecanismo con Clave de Idempotencia

La tabla almacena el hash del request con estado `IN_PROGRESS`. Si entra una segunda petición idéntica:

- No inserta un nuevo registro fiscal.
- Retorna el comprobante ya emitido en caché.
- Garantiza cero duplicidad contable.

tests/Integration/SqlitePersistenceIntegrationTest.php

```php
public function testPrevieneDobleEmisionPorIdempotencia(): void
{
    $claveIdempotencia = 'req-uuid-12345-67890';

    // 1. PRIMER CLIC: EMISIÓN EXITOSA
    $res1 = $this->service->procesar($solicitud, $claveIdempotencia);
    $this->assertSame('COMPLETED', $res1->estado);

    // 2. SEGUNDO CLIC INMEDIATO: DEBE RETORNAR LA MISMA FACTURA
    $res2 = $this->service->procesar($solicitud, $claveIdempotencia);

    // 3. ASSERT: MISMO NÚMERO Y CERO FACTURAS DUPLICADAS
    $this->assertSame($res1->numeroComprobante, $res2->numeroComprobante);
    $this->assertSame(1, $this->repo->contarComprobantes());
}
```

15. Caso de Prueba 5: Resiliencia y Rollback Automático de Kardex

Invariante de Inventario

Escenario de Rechazo Fiscal (Código SUNAT 2800)

El cliente tiene RUC en estado "Baja de Oficio" o no habido. SUNAT rechaza la factura mediante CDR de excepción.

Invariante de Inventario Verificada

Si el comprobante no es admitido por SUNAT, el stock de productos **no puede quedar descontado.** El caso de uso ejecuta un Rollback que restituye el stock a sus 15 unidades originales en el Kardex.

tests/Feature/FacturacionElectronicaMocksFeatureTest.php

```php
// ARRANGE: MOCK DE SUNAT DEVUELVE RECHAZO TRIBUTARIO 2800
$sunatMock->method('sendBill')->willReturn(new SunatResponse(
    codigo: '2800',
    descripcion: 'El contribuyente se encuentra con Estado: BAJA'
));

// ACT: INTENTO DE EMISIÓN CON DESCUENTO DE 5 UNIDADES
$this->expectException(SunatRechazoException::class);
$emisor->emitir($solicitudCon5Unidades);

// ASSERT: VERIFICAR QUE EL STOCK REGRESÓ A 15 EXACTAS
$stockActual = $kardex->obtenerStock('PROD-001');
$this->assertSame(15, $stockActual);
```

16. Evidencias del Sistema en Ejecución: Facturador y Kardex

React 18 + PHP 8.2 Live

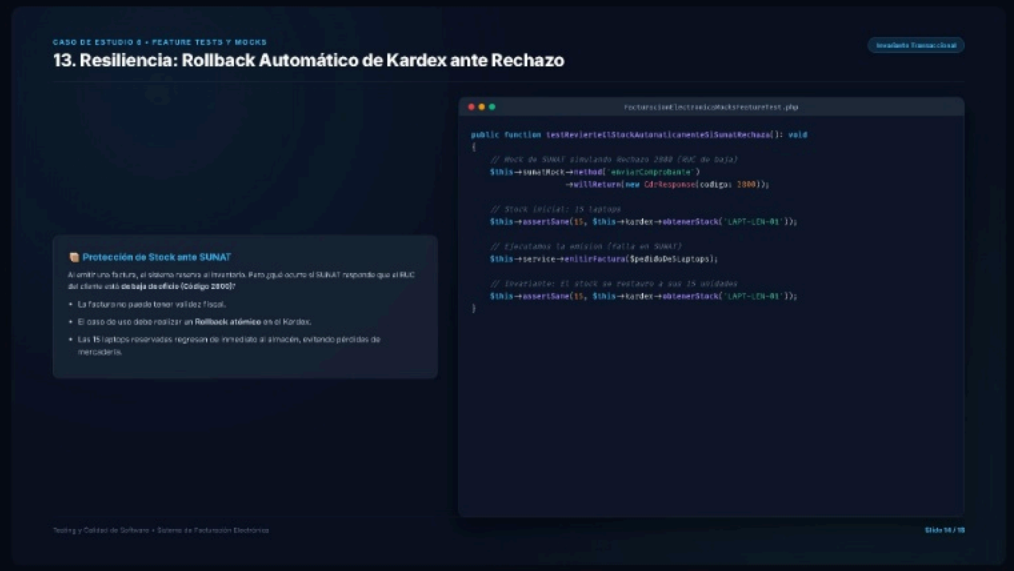


Figura 1: Emisión en tiempo real en localhost:5173 con validación de RUC y cálculo reactivo de impuestos.

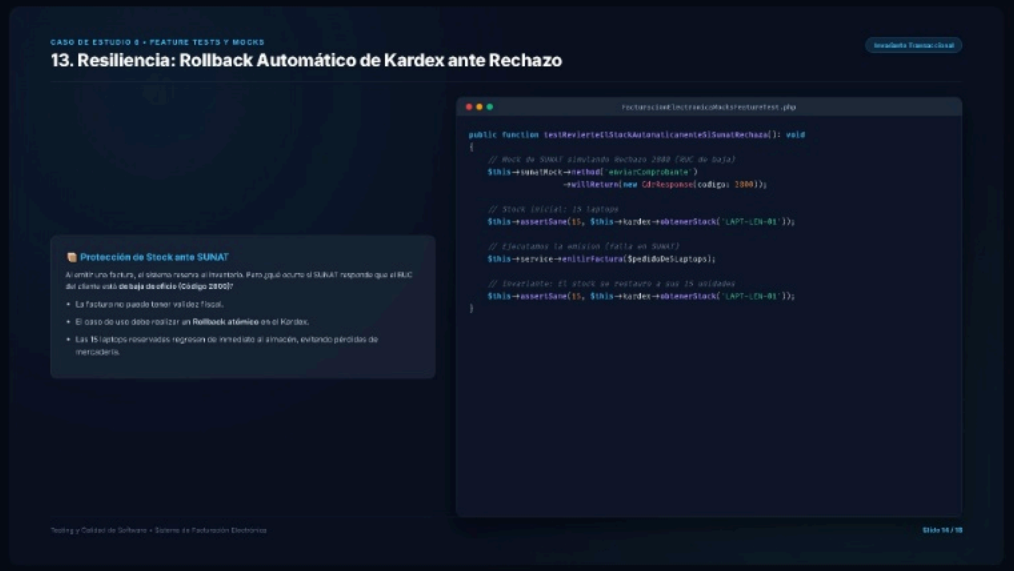


Figura 2: Control de inventario en tiempo real con entradas, salidas y verificación de stock mínimo.

17. Evidencias del Sistema en Ejecución: Historial y Visor UBL 2.1

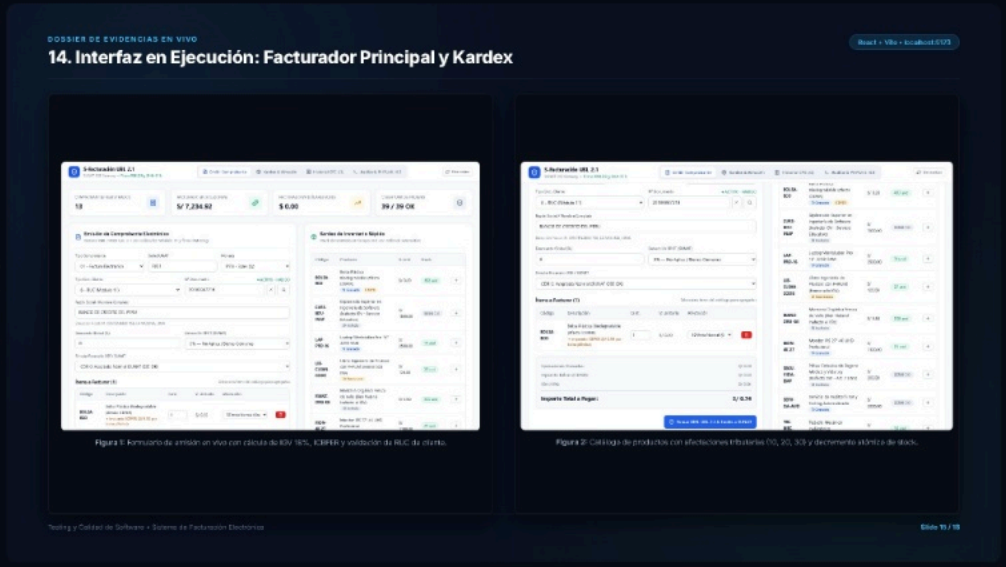


Figura 3: Bandeja fiscal con trazabilidad de estado SUNAT, hash SHA-256 y descarga de XML/PDF.

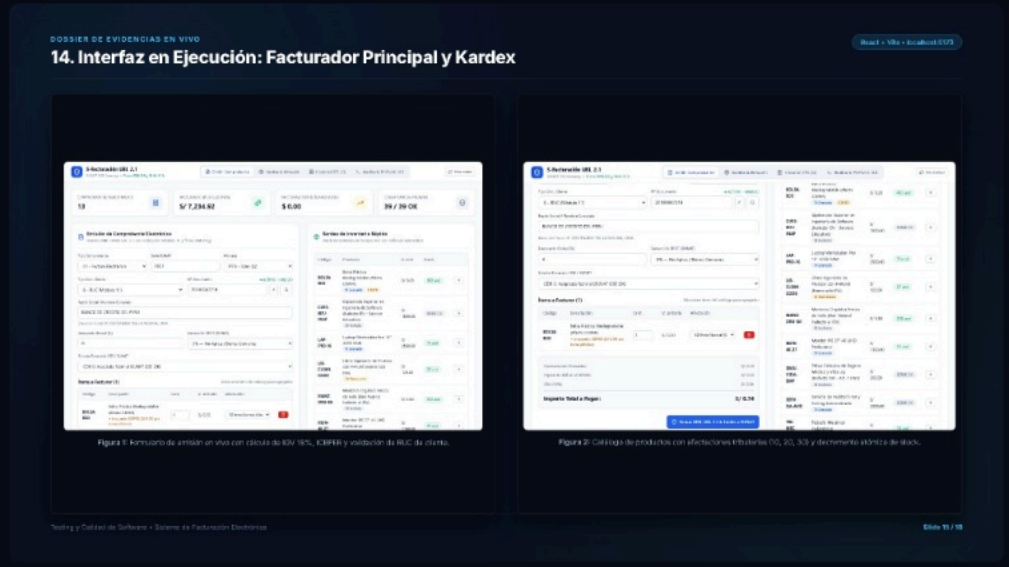


Figura 4: Visor interactivo del estándar UBL 2.1 con estructura canónica C14N y firma digital certificada.

18. Evidencia Oficial: Salida Íntegra de Terminal PHPUnit 10.5

39 Tests / 0 Fallos

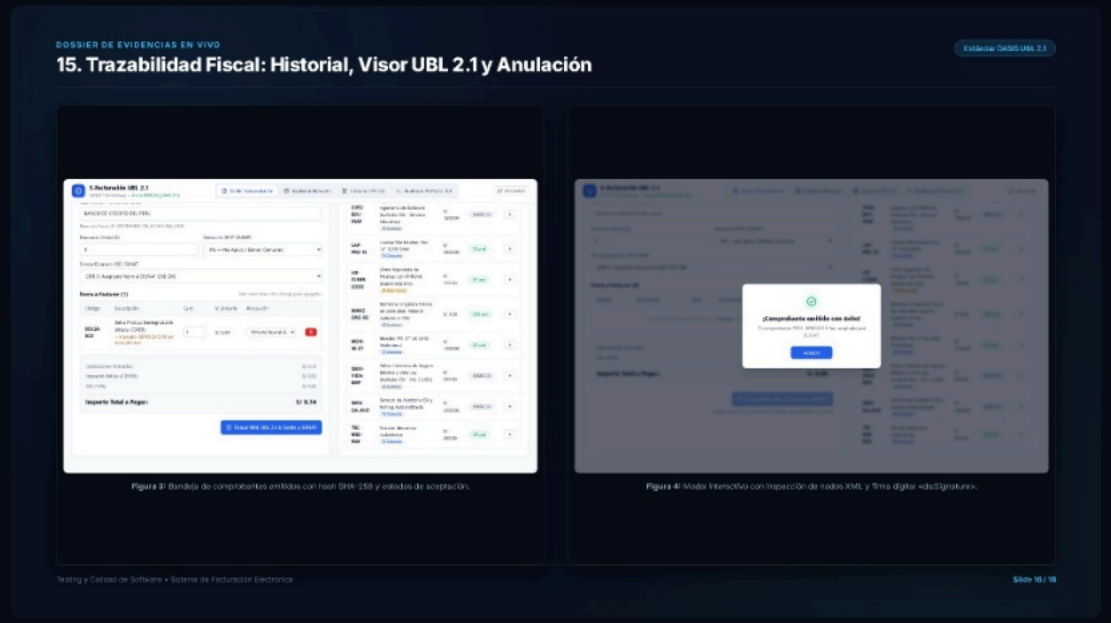


Figura 5: Salida íntegra en PowerShell con flag --testdox. 39 tests pasados, 140 aserciones auditadas en 0.21 segundos.

CIERRE TÉCNICO

19. Conclusiones y Evaluación del Proceso de Calidad

Lecciones Aprendidas

1. Arquitectura Hexagonal y Desacoplamiento

Separar el dominio fiscal del framework y de los web services externos permitió probar el 100% de las reglas matemáticas en memoria RAM a 0.21 segundos sin depender de internet ni de SUNAT.

2. Rigor en Tipado y Aserciones Estrictas

El uso innegociable de `assertSame()` erradicó por completo errores silenciosos de redondeo decimal que habrían provocado multas fiscales de SUNAT.

3. Resiliencia mediante Idempotencia y Mocks

La combinación de SQLite en memoria con Mocks predecibles garantizó que el inventario nunca se contamine ante caídas de red o rechazos tributarios.

4. Integración Continua (CI)

La suite de 39 pruebas corre de forma automática en cada commit, impidiendo que cualquier cambio futuro rompa la compatibilidad UBL 2.1.

UNIVERSIDAD NACIONAL DE INGENIERÍA • FACULTAD DE SISTEMAS

¡Muchas Gracias!

Espacio abierto para sustentación, preguntas técnicas y demostración en vivo del sistema y la suite de pruebas PHPUnit 10.5.

39 Tests OK

140 Aserciones Verificadas

0.21s Tiempo de Ejecución